

Human-Centered Controller Mapping Background

How Should the Controller Match the Driver's Intuition?

Robot control is not only a programming problem. It is also a human design problem.

When drivers are calm, they can think carefully about what each button does. When drivers are stressed, rushed, defended, or trying to score before time runs out, they rely more on habit, muscle memory, and previous game experience.

That means a good controller map should feel predictable. The best control scheme is not always the one with the most features. It is the one the driver can use correctly under pressure.

The Big Idea

A controller map is a design decision. Teams should choose controls that match:

- what the robot needs to do,
- what the driver already understands,
- what actions are most urgent,
- what mistakes would be costly,
- and what the team can practice enough to make automatic.

If the driver has to stop and remember the controls during a match, the map is too complicated or not practiced enough.

Human Intuition and Stress

Under pressure, people often fall back on familiar patterns.

Examples:

- If a student has played console games, they may expect the left stick to move and the right stick to turn or aim.
- If a student has used D-pad controls in simple games, they may expect each direction to mean a clear fixed movement.
- If a student has driven remote-control cars, they may expect one stick or trigger to control forward/backward and another control to steer.
- If a student has practiced the same loading and shooting routine many times, they may do it smoothly without needing much verbal coaching.

This is why practice matters. A confusing control map can become manageable with training, but a simple map becomes powerful when the driver has practiced it repeatedly.

What Is the Logitech F310?

The Logitech F310 is a wired USB gamepad commonly used with FTC Driver Stations. It is designed mainly as a PC game controller.

Important features:

Feature	Why It Matters
USB wired connection	Simple and reliable for Driver Station use.
XInput mode	Makes the controller behave like a modern Windows/Xbox-style gamepad.
DirectInput mode	Older PC input mode; useful in some computer games, but usually not the first choice for FTC.
Two analog sticks	Useful for smooth driving, turning, aiming, or mechanism control.
D-pad	Useful for simple fixed-direction driving or step-by-step adjustments.
A/B/X/Y buttons	Useful for discrete actions such as shoot, open trigger, reset, slow mode, or mode change.
Bumpers and triggers	Useful for actions the driver may need while keeping thumbs on the sticks.

For FTC use, the switch on the bottom of the F310 is usually set to **X** for XInput mode. In many FTC setup examples, the driver assigns the controller by pressing **Start + A** for gamepad 1 or **Start + B** for gamepad 2.

How the F310 Relates to Xbox, PlayStation, and PC Controllers

The F310 is not an Xbox or PlayStation controller, but it borrows familiar ideas from both console-style and PC gamepads.

Controller Family	Connection to the F310
PC gamepads	The F310 is officially a PC-style USB gamepad and supports common PC input standards.
Xbox-style controllers	In XInput mode, the F310 uses Xbox-style button names: A, B, X, Y, bumpers, triggers, sticks, Start, and Back. FTC code also commonly uses these names.
PlayStation-style controllers	The physical idea is familiar: two sticks, a D-pad, four face buttons, shoulder controls, and center buttons. The labels are different: PlayStation uses symbols such as triangle, circle, cross, and square instead of A/B/X/Y.
Nintendo-style controllers	The F310 also has four face buttons and a D-pad, but button positions and labels may not match what a Nintendo player expects.

This matters because students may bring different controller habits. A driver who grew up with PlayStation may understand the layout but not the A/B/X/Y names automatically. A driver used to Xbox-style games may recognize the names more quickly.

FTC Code Names

In FTC programs, controller inputs are usually read through names like:

Driver Action	Common FTC Code Name
Left stick forward/back	<code>gamepad1.left_stick_y</code>

Driver Action	Common FTC Code Name
Left stick left/right	<code>gamepad1.left_stick_x</code>
Right stick left/right	<code>gamepad1.right_stick_x</code>
D-pad directions	<code>gamepad1.dpad_up</code> , <code>gamepad1.dpad_down</code> , <code>gamepad1.dpad_left</code> , <code>gamepad1.dpad_right</code>
Face buttons	<code>gamepad1.a</code> , <code>gamepad1.b</code> , <code>gamepad1.x</code> , <code>gamepad1.y</code>
Bumpers	<code>gamepad1.left_bumper</code> , <code>gamepad1.right_bumper</code>
Triggers	<code>gamepad1.left_trigger</code> , <code>gamepad1.right_trigger</code>

This means the code should be written in a way the team can explain. For example:

- A might mean shoot.
- B might mean stop shooter.
- Right bumper might open the trigger.
- Left bumper might close the trigger.
- Right trigger might activate slow mode.

The exact map is a team decision.

D-Pad Control

D-pad control uses the cross-shaped direction pad.

What It Helps	What It Makes Harder
Simple directions	Smooth curves
Beginner predictability	Fine speed control
Testing one movement at a time	Natural driving feel
Repeatable short moves	Fast aiming adjustments

D-pad control can be good for first tests because it is clear: up means forward, down means backward, left means turn or slide left, and right means turn or slide right. It can also feel less natural when the driver needs smooth aiming or tiny corrections.

Joystick Control

Joystick control uses one or both analog sticks.

What It Helps	What It Makes Harder
Smooth driving	Beginner overcorrection
Variable speed	Consistent straight-line movement
Natural video-game feel	Debugging if directions are reversed
Fast aiming corrections	Avoiding sudden full-power movement

Joystick control often feels more like modern console or PC games. It can be powerful, but it may need dead zones, speed scaling, or a precision mode so the robot does not jump or turn too sharply.

Speed and Precision Modes

A robot that is easy to drive at full speed may still be hard to aim. Teams can program control modes to help.

Mode	What It Does	When It Helps
Normal mode	Full driver control range	Moving across the field
Slow mode	Reduces motor power	Aiming, loading, parking, avoiding penalties
Precision turn	Reduces turning speed	Lining up with a launch site
Dead zone	Ignores tiny stick movements	Prevents drift when the stick is near center

Speed modes should be easy to remember. If slow mode is important, put it on a control the driver can reach without looking down.

Mapping Mechanisms

Driving controls should be the easiest to use because the robot is moving while the driver is thinking. Mechanism controls should also match the team's routine.

For a human-loaded ball launcher, the sequence might be:

- Drive to loading position.
- Human operator loads up to three balls.
- Driver moves to launch site.
- Driver aims.
- Driver spins up flywheel.
- Driver opens trigger to release one ball or a controlled set of balls.
- Driver repeats or returns to base.

The controller map should support this sequence.

Robot Action	Human-Centered Mapping Question
Drive	Can the driver move without looking down?
Slow/precision mode	Can the driver aim without accidentally driving too fast?
Spin up flywheel	Is the control hard to press by accident?
Open trigger	Is the control easy to press at the exact moment needed?
Stop shooter	Is there a clear safe stop action?
Reset or unjam	Is this protected from accidental use?

Safety and Error Prevention

Some controls should be easy. Some controls should be deliberate.

Control Type	Good Design Choice
Emergency stop or safe stop	Easy to find and easy to explain.
Driving	Natural and practiced.
Precision mode	Easy to hold or toggle while aiming.
Shooter spin-up	Deliberate enough to avoid accidental activation.
Trigger release	Clear and consistent.
Reset or reverse	Protected from accidental presses.

A good question is: "What is the worst mistake the driver could make with this button map?" Then design the controls to make that mistake less likely.

Training Matters

A good controller map is only good if the driver practices it.

Teams should test:

- Can the driver explain every control without looking?
- Can the driver complete a loading, driving, aiming, and shooting cycle three times?
- Does the driver accidentally press the wrong button?
- Does the driver overcorrect while aiming?
- Does the human operator understand the driver's signals?
- Can another teammate use the same map if the main driver is absent?

Main Takeaway

Controller mapping is a human-centered design decision. The code should match the driver, the robot, the game strategy, and the team's practice time.

The best explanation sounds like this:

We chose this controller map because it matched our driver's intuition, reduced mistakes under pressure, and helped us complete our scoring routine reliably.

Source Notes

This page is based on FTC controller programming conventions and Logitech F310 documentation. Logitech describes the F310 as a PC gamepad with XInput and DirectInput modes. FTC setup examples commonly use the F310 in XInput mode, which behaves like an Xbox-style controller and uses familiar A/B/X/Y button names.